NS3D — Numerical Methods and Discretization

Steady, low-Reynolds incompressible flow in 3-D porous media
(SIMPLE / SIMPLEC / SIMPLER / IDEAL / SIMPLER-R)

J. Lux

October 13, 2026

Abstract

This document gives a self-contained description of the numerical methods implemented in `NS3D`: the steady, low-Reynolds incompressible Navier–Stokes equations discretized with a finite-volume method on a staggered (MAC) grid, and the pressure–velocity coupling algorithms Simple, Simplec, Simpler, Ideal and Simpler-R. The discrete momentum equation, the diffusive and convective fluxes, the under-relaxation, the boundary conditions and the derivation of the pressure correction are all written out explicitly. The document is faithful to the reference C++ implementation in `src/`.

Contents

1 Governing equations **2**

2 Discretization **2**
2.1 Staggered (MAC) grid . . . 2
2.2 Finite-volume form . . . 2
2.3 Diffusive fluxes . . . 2
2.4 Convective fluxes . . . 3
2.5 Assembled coefficients . . . 3

3 Under-relaxation **3**

4 Boundary conditions **4**

5 Pressure–velocity coupling **4**
5.1 Derivation of the correction relation . . . 4
5.2 Simple . . . 4
5.3 Simplec . . . 5
5.4 Simpler . . . 5
5.5 Ideal . . . 6
5.6 Simpler-R . . . 6
5.7 Summary of the schemes . . . 6

6 Convergence control **6**
6.1 Adaptive relaxation . . . 7
6.2 Velocity limiting . . . 7

7 Derived quantities **7**
7.1 Permeability (Darcy) . . . 7
7.2 Forchheimer . . . 7
7.3 Dimensionless permeability . . . 7

Notation 8

References 8

1 Governing equations

The flow is assumed incompressible, steady and at low Reynolds number. The continuity and momentum equations read

$$\nabla\cdot\mathbf{u} = 0, \tag{1}$$

$$\rho\,\mathbf{u}\cdot\nabla\,\mathbf{u} = -\nabla p + \mu\,\nabla^2\mathbf{u}, \tag{2}$$

where $\mathbf{u} = (u, v, w)$ is the velocity, p the pressure, ρ the density and μ the dynamic viscosity. In component form the x-momentum equation is

$$\mu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}\right) - \rho\left(u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} + w\frac{\partial u}{\partial z}\right) = \frac{\partial p}{\partial x}, \tag{3}$$

with analogous expressions for the y and z components.

2 Discretization

2.1 Staggered (MAC) grid

A staggered grid is used: the pressure p is stored at cell centres and the three velocity components are stored on the cell faces, one component per face. A pressure node C has six face neighbours, denoted W, E (along x), S, N (along y) and B, T (along z). On the face between C and E the u-component lives, on the face between C and N the v-component lives, and so on. The mesh is assumed Cartesian and locally uniform with spacings $\Delta x, \Delta y, \Delta z$.

2.2 Finite-volume form

Equation (2) is integrated over a control volume V bounded by S. Applying the divergence theorem to the convection and diffusion terms gives, for the u-component,

$$\frac{1}{V}\int_S \mu\,\nabla u\cdot\mathbf{n}\,\mathrm{d}S - \frac{1}{V}\int_S \rho\,(\mathbf{u}\cdot\mathbf{n})\,u\,\mathrm{d}S = \frac{1}{V}\int_V \frac{\partial p}{\partial x}\,\mathrm{d}V. \tag{4}$$

Writing the surface integrals as the sum of the contributions of the six faces, the discrete equation for the u-node C takes the standard linear form

$$a_{\mathrm{C}}^{u}\,u_{\mathrm{C}} + \sum_{n\in\{\mathrm{W,E,S,N,B,T}\}} a_n^{u}\,u_n = \frac{p_{\mathrm{C}} - p_{\mathrm{W}}}{\Delta x} + b, \tag{5}$$

where the coefficients a_n^u are built from the diffusive and convective fluxes on face n (Sections 2.3–2.4), and the right-hand side is the imposed pressure gradient (or the boundary value). Equations of the same form hold for v (pressure gradient $p_{\mathrm{C}} - p_{\mathrm{S}}$ over Δy) and w (over Δz).

2.3 Diffusive fluxes

The diffusive contribution on a face is a second-order central difference. For the u-node C:

$$D_{\mathrm{W}}^{u} = \mu\frac{u_{\mathrm{C}} - u_{\mathrm{W}}}{\Delta x^2}, \qquad D_{\mathrm{E}}^{u} = \mu\frac{u_{\mathrm{E}} - u_{\mathrm{C}}}{\Delta x^2}, \tag{6}$$

$$D_{\mathrm{S}}^{u} = \mu\frac{u_{\mathrm{C}} - u_{\mathrm{S}}}{\Delta y^2}, \qquad D_{\mathrm{N}}^{u} = \mu\frac{u_{\mathrm{N}} - u_{\mathrm{C}}}{\Delta y^2}, \tag{7}$$

$$D_{\mathrm{B}}^{u} = \mu\frac{u_{\mathrm{C}} - u_{\mathrm{B}}}{\Delta z^2}, \qquad D_{\mathrm{T}}^{u} = \mu\frac{u_{\mathrm{T}} - u_{\mathrm{C}}}{\Delta z^2}. \tag{8}$$

2.4 Convective fluxes

Let u^*, v^*, w^* denote the velocity field from the previous iteration (the coefficients are lagged). The convective flux on a face is either *centered* or *upwind*, selected by a local Péclet criterion. Writing $D = \mu/\Delta x^2$ (or $\mu/\Delta y^2$, $\mu/\Delta z^2$) and F for the signed face flux, the switch implemented in `NSBase::AssembleMomentum_*` is

$$|F|\,\Delta x^2 \;>\; 2\,\mu\,s_{\text{sch}} \quad\Longrightarrow\quad \text{upwind}, \qquad \text{otherwise centered}, \tag{9}$$

where s_{sch} is the *scheme factor*. With the default $s_{\text{sch}} = 0$ the test reduces to $|F| > 0$, i.e. the upwind scheme is always used whenever the face flux is non-zero; the centered scheme is taken only for a strictly zero flux. This is the intended behaviour for the low-Reynolds flows solved here (an always-upwind convection is unconditionally stable).

The upwind coefficients on a face n are

$$a_{\text{C}}^{u} \;+\!=\; -D + \min(F, 0), \qquad a_{n}^{u} \;=\; D + \max(F, 0), \tag{10}$$

and the centered coefficients are

$$a_{\text{C}}^{u} \;+\!=\; -D + \tfrac{1}{2}F, \qquad a_{n}^{u} \;=\; D + \tfrac{1}{2}F. \tag{11}$$

Here F is the signed convective flux (positive for inflow through the face). For the two faces along the u-axis the flux is built from the u-values on either side of the face; for the transverse faces the face velocity is the average of the two v (resp. w) values flanking the face, e.g. for the N face of node C, $v_{\text{N}} = \tfrac{1}{2}\big(v_{\text{C}-} + v_{\text{C}}\big)$ and $F = -v_{\text{N}}\,\rho/\Delta y$.

2.5 Assembled coefficients

For the u-node C the diagonal coefficient is the sum over the six faces of the central part,

$$a_{\text{C}}^{u} = \sum_{n} D_{n}^{u} \;-\; \sum_{n} \min(F_n, 0) \qquad \text{(upwind)}, \tag{12}$$

and each off-diagonal is given by (10) or (11). The v and w systems are identical with the indices shifted. The three momentum systems are assembled as sparse matrices $\mathbf{A}^u, \mathbf{A}^v, \mathbf{A}^w$ and factorized with an iterative linear solver.

3 Under-relaxation

The u-update is relaxed with a factor $\alpha \in (0, 1]$. Writing the unrelaxed equation (5) as

$$a_{\text{C}}^{u}\,u_{\text{C}}^{n} + \sum_{n} a_{n}^{u}\,u_{n}^{n} = \frac{p_{\text{C}} - p_{\text{W}}}{\Delta x} + b, \tag{13}$$

and imposing $u^{n} - u^{n-1} = \alpha\,$(correction), one obtains the relaxed system solved by the code:

$$\boxed{\frac{a_{\text{C}}^{u}}{\alpha}\,u_{\text{C}}^{n} + \sum_{n} a_{n}^{u}\,u_{n}^{n} = \frac{p_{\text{C}} - p_{\text{W}}}{\Delta x} + b + \frac{1-\alpha}{\alpha}\,a_{\text{C}}^{u}\,u_{\text{C}}^{n-1}} \tag{14}$$

In the implementation only the diagonal is scaled by $1/\alpha$ (`A(c,c) /= alpha`); the right-hand side is augmented with `u(c) * A(c,c) * (1-alpha)` (see `Assemble_RHS_*`). A separate pressure under-relaxation factor β is applied to the pressure equation and to the pressure update (Section 5).

4 Boundary conditions

- **No-slip solid/fluid interface.** The velocity is zero at the face. The convective flux vanishes, but the diffusive flux is retained with a halved distance, which doubles the diagonal contribution on that face:
$$a_{\mathrm{C}}^{u} \;+\!= \; -2D \qquad \text{(solid face)}. \tag{15}$$
- **Inlet.** The velocity is imposed; only the component along the flow direction is non-zero. The pressure is fixed to p_0, so the right-hand side reads $(p_{\mathrm{C}} - p_0)/\Delta x$.
- **Outlet.** The velocity is extrapolated (zero gradient) and the pressure is fixed to p_1; the right-hand side reads $(p_1 - p_{\mathrm{W}})/\Delta x$.

5 Pressure–velocity coupling

5.1 Derivation of the correction relation

The momentum equation written for a *guess* velocity $\mathbf{u}^*$ is

$$\frac{a_{\mathrm{C}}^{u}}{\alpha}\, u_{\mathrm{C}}^{*} + \sum_{n} a_{n}^{u}\, u_{n}^{*} = \frac{p_{\mathrm{C}}^{(n-1)} - p_{\mathrm{W}}^{(n-1)}}{\Delta x} + b + \frac{1-\alpha}{\alpha}\, a_{\mathrm{C}}^{u}\, u_{\mathrm{C}}^{(n-1)}. \tag{16}$$

Define the *pseudo-velocity* (or *H-function*)

$$\widehat{u}_{\mathrm{C}} = -\frac{\sum_n a_n^u\, u_n^*}{a_{\mathrm{C}}^u/\alpha} + \frac{1-\alpha}{\alpha}\, a_{\mathrm{C}}^{u}\, u_{\mathrm{C}}^{(n-1)} \;=\; -\frac{\alpha}{a_{\mathrm{C}}^{u}} \sum_{n} a_{n}^{u}\, u_{n}^{*} + (1-\alpha)\, u_{\mathrm{C}}^{(n-1)}, \tag{17}$$

so that (16) becomes the simple, pressure-only relation

$$\boxed{u_{\mathrm{C}}^{*} = \widehat{u}_{\mathrm{C}} + \frac{1}{a_{\mathrm{C}}^{u}/\alpha}\, \frac{p_{\mathrm{C}} - p_{\mathrm{W}}}{\Delta x}} \tag{18}$$

which the code computes in `Compute_{u,v,w}_hat` and `CorrectVelocityFieldSIMPLER`.

The actual (corrected) velocity $\mathbf{u} = \mathbf{u}^* + \mathbf{u}'$ is required to satisfy both the momentum equation and the continuity equation. Subtracting (16) from the equation for $\mathbf{u}$ and using $\nabla\cdot\mathbf{u}' = \nabla\cdot\mathbf{u} = 0$ gives, to first order,

$$\frac{a_{\mathrm{C}}^{u}}{\alpha}\, u_{\mathrm{C}}' + \sum_{n} a_{n}^{u}\, u_{n}' = \frac{p_{\mathrm{C}}' - p_{\mathrm{W}}'}{\Delta x}. \tag{19}$$

Each algorithm differs in how the sum over the neighbours $\sum_n a_n^u u_n'$ in (19) is approximated.

5.2 Simple

The Simple algorithm (Semi-Implicit Method for Pressure Linked Equations) was introduced by Patankar and Spalding [1, 2]. The neighbour correction is neglected, $\sum_n a_n^u u_n' \approx 0$, so that the face correction is

$$u_{\mathrm{C}}' = \frac{1}{a_{\mathrm{C}}^{u}/\alpha}\, \frac{p_{\mathrm{C}}' - p_{\mathrm{W}}'}{\Delta x} \qquad \Longleftrightarrow \qquad u_{\mathrm{C}}' = \frac{\alpha}{a_{\mathrm{C}}^{u}}\, \frac{p_{\mathrm{C}}' - p_{\mathrm{W}}'}{\Delta x}. \tag{20}$$

Substituting (20) into the continuity equation $\nabla\cdot\mathbf{u}^* = 0$ yields the pressure-correction Poisson system. With

$$d_{\mathrm{C}}^{u} = \frac{1}{(a_{\mathrm{C}}^{u}/\alpha)\,\Delta x^2}, \tag{21}$$

the node equation is

$$\sum_n \left[d_{\mathrm{C}}^u p'_{\mathrm{C}} - d_{\mathrm{C}}^u p'_n \right] = \sum_n \frac{u^*_{\mathrm{face}}}{\Delta x}, \tag{22}$$

i.e. a five-point (six-point in 3-D) Laplacian with coefficients $d_{\mathrm{C}}^u, d_{\mathrm{C}}^v, d_{\mathrm{C}}^w$ and a source made of the divergence of the guessed velocity. This is `AssemblePressureCorrectionSIMPLE` and `CorrectVelocityFieldSIMPLE`.

5.3 SIMPLEC

SIMPLEC (SIMPLE-consistent) was proposed by Manasria and Mudawar [3]. The neglected sum in (19) is approximated by the central value,

$$\sum_n a_n^u u'_n \;\approx\; u'_{\mathrm{C}} \sum_n a_n^u, \tag{23}$$

which gives

$$\boxed{u'_{\mathrm{C}} = \frac{1}{\dfrac{a_{\mathrm{C}}^u}{\alpha} + \displaystyle\sum_n a_n^u} \frac{p'_{\mathrm{C}} - p'_{\mathrm{W}}}{\Delta x}} \tag{24}$$

Note that $\dfrac{a_{\mathrm{C}}^u}{\alpha} + \sum_n a_n^u$ is exactly the *row sum* of the momentum matrix row C, which is what the code uses (`A.row(c).sum()`). The pressure-correction coefficients are therefore

$$d_{\mathrm{C}}^u = \frac{1}{\Delta x^2} \left[\frac{a_{\mathrm{C}}^u}{\alpha} + \sum_n a_n^u \right]^{-1}, \tag{25}$$

assembled in `AssemblePressureCorrectionSIMPLEC` and applied in `CorrectVelocityFieldSIMPLEC`.

5.4 SIMPLER

SIMPLER (SIMPLE Reformulation) was introduced by Acharya [4]. It splits the update into a pressure step and a velocity correction step, using the pseudo-velocity (18). One outer iteration is:

1. **Pressure step** (inner loop, N_1 times): from the current velocity compute the pseudo-velocities $\widehat{u}, \widehat{v}, \widehat{w}$ ((17)) and solve the continuity system (with the SIMPLE coefficients of (20)) for a new pressure p^*. Then update the working velocity using (18) (`AssemblePressure` + `CorrectVelocityFieldSIMPLER`).

2. **Velocity step**: solve the relaxed momentum system (14) with the new pressure p^* to obtain the velocity $\mathbf{u}^*$ (`ComputeVelocityField`).

3. **Correction step**: assemble and solve the SIMPLE pressure-correction system (20) for p' and apply (20) to the velocity only (`CorrectVelocityFieldSIMPLE`). The pressure itself is *not* updated with p'.

Extra memory: the pseudo-velocities $\widehat{u}, \widehat{v}, \widehat{w}$.

5.5 IDEAL

IDEAL (Inner Doubly Iterative Efficient Algorithm for Linked Equations) was proposed by Sun *et al.* [5]. It keeps two pressure solves and one momentum solve, and updates the pressure at the end of the iteration (unlike SIMPLER). One outer iteration is:

1. **Pressure step 1** (inner loop, N_1 times): compute pseudo-velocities from the working velocity, solve the continuity system for p^* and update the working velocity with (18).
2. **Velocity step**: solve the relaxed momentum system with p^* to obtain $\mathbf{u}^*$.
3. **Pressure step 2** (inner loop, N_2 times): compute the pseudo-velocities from $\mathbf{u}^*$, solve the continuity system again (this time *without* pressure under-relaxation) for p^{**} and update the velocity with (18). The final pressure is p^{**}.

The advantage over SIMPLER is that the final pressure is obtained by a direct continuity solve rather than by a pressure-correction solve, which is typically better conditioned.

5.6 SIMPLER-R

SIMPLER-R (SIMPLER-revised) [6] is a hybrid of IDEAL and SIMPLER:

1. **Pressure step 1** (inner loop, N_1 times): as in IDEAL, solve the continuity system for p^* and update the working velocity with (18).
2. **Velocity step**: solve the relaxed momentum system with p^* to obtain $\mathbf{u}^*$.
3. **Correction step**: solve the SIMPLE pressure-correction system (20) for p' and apply (20) to the velocity.
4. **Pressure refresh**: recompute the pseudo-velocities from the corrected velocity, then solve the continuity system once more (with pressure under-relaxation) to update the pressure p.

This combines the velocity correction of SIMPLER with the pressure refresh of IDEAL.

5.7 Summary of the schemes

A direct numerical comparison of PISO, SIMPLER and SIMPLEC for steady incompressible flow was performed by Jang *et al.* [7]; the relative cost and conditioning of the pressure solves summarized below is consistent with that study.

Scheme	# pressure solves	Momentum solve	Velocity correction	Pressure corrected
SIMPLE	1	1	(20) (diag.)	yes ($\beta\, p'$)
SIMPLEC	1	1	(24) (row sum)	yes ($\beta\, p'$)
SIMPLER	$N_1 + 1$	1	(20) (diag.)	no
IDEAL	$N_1 + N_2$	1	(18)	yes (direct)
SIMPLER-R	$N_1 + 2$	1	(20) (diag.)	yes (direct)

Table 1: Comparison of the five pressure–velocity coupling schemes. The "# pressure solves" counts the continuity solves per outer iteration (including the inner N_1/N_2 loops).

6 Convergence control

Two mechanisms, both implemented in `NavierStokesBase::RelaxControl` and the velocity-correction functions, are used to keep the iteration stable.

6.1 Adaptive relaxation

Let RMR be the relative mass residual. The relaxation factors are adapted as follows:

- **Divergence** ($\mathrm{RMR} > \mathrm{RMR}_{\mathrm{old}}\,\delta_{\max}$): decrease α (and β unless it is held constant), $\alpha \leftarrow \max(\alpha_{\min}, \alpha - \Delta)$, and reset $\mathrm{RMR}_{\mathrm{old}} \leftarrow 1.1\,\mathrm{RMR}_{\mathrm{old}}$.
- **Oscillation** (the recent average residual is flat): decrease α, β by a smaller increment ($\Delta/4$).
- **Slow divergence**: decrease α, β by Δ.

6.2 Velocity limiting

After each velocity solve/correction, the per-node velocity change is clamped to a maximum increment $\delta v_{\max}$. In the code $\delta v_{\max} = v_{\mathrm{moy}} \min(1, \alpha\,1.5)$ for the momentum solve and a large value ($100\,v_{\mathrm{moy}}$, i.e. effectively unbounded) for the correction steps, so that only the main momentum update is limited.

7 Derived quantities

7.1 Permeability (Darcy)

For a fully-developed, unidirectional flow the Darcy law $-\nabla\langle p\rangle = \mu\,\mathbf{K}^{-1}\langle \mathbf{u}\rangle$ is used to define the permeability from the computed mean velocity $\langle v\rangle$ and the imposed pressure difference:

$$K = \frac{\langle v\rangle\,\mu\,\Delta x\,\big(N_x - 2\,b\big)}{p_{\mathrm{in}} - p_{\mathrm{out}}}, \tag{26}$$

where N_x is the number of cells along the flow direction and b the number of solid border cells. The mean velocity is obtained by averaging the velocity over the inlet/outlet faces. The code guards against division by zero when $p_{\mathrm{in}} = p_{\mathrm{out}}$.

7.2 Forchheimer

For flows where inertia is non-negligible, the macroscopic momentum balance is the Forchheimer (or Darcy–Forchheimer) relation

$$-\nabla\langle p\rangle = \frac{\mu}{k_1}\,\langle v\rangle + \frac{\rho}{k_2}\,\langle v\rangle^2, \tag{27}$$

with the linear coefficient k_1 (permeability) and the inertial coefficient k_2. The two coefficients are identified by fitting the computed $(\Delta p, \langle v\rangle)$ data to this quadratic.

7.3 Dimensionless permeability

Defining the reference quantities L_{ref} (length), v_{ref} (velocity) and $\Delta\langle P^*\rangle$ (dimensionless pressure drop), the dimensional permeability is

$$K = \frac{\langle v^*\rangle\,\Delta L^*}{\epsilon\,\Delta\langle P^*\rangle}\,\frac{\mu\,L_{\mathrm{ref}}}{\rho\,v_{\mathrm{ref}}} = \frac{L_{\mathrm{ref}}^2}{\mathrm{Re}}\,K^*, \tag{28}$$

where $\mathrm{Re} = \rho v_{\mathrm{ref}} L_{\mathrm{ref}}/\mu$ and ϵ the porosity.

Notation

Symbol	Meaning
$\mathbf{u} = (u, v, w)$	velocity vector
p	pressure
ρ, μ	density, dynamic viscosity
α, β	velocity, pressure under-relaxation factors
$\mathrm{C}, \mathrm{W}, \mathrm{E}, \mathrm{S}, \mathrm{N}, \mathrm{B}, \mathrm{T}$	cell centre and face neighbours
a_{C}^{u}	diagonal coefficient of the u-momentum equation
a_{n}^{u}	off-diagonal coefficient towards neighbour n
D_{n}^{u}	diffusive flux on face n
F_n	signed convective flux on face n
$\widehat{u}$	pseudo-velocity (H-function)
p'	pressure correction
d_{C}^{u}	pressure-correction coefficient for u
N_1, N_2	inner-loop counts (SIMPLER/IDEAL)
RMR	relative mass residual
K, k_1, k_2	permeability, Forchheimer coefficients
$\Delta x, \Delta y, \Delta z$	mesh spacings

References

[1] S. V. Patankar and D. B. Spalding, "A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows," *Int. J. Heat Mass Transfer*, vol. 15, no. 10, pp. 1787–1806, 1972. doi: 10.1016/0017-9310(72)90054-3.

[2] S. V. Patankar, *Numerical Heat Transfer and Fluid Flow*, Hemisphere Publishing, Washington, D.C., 1980.

[3] R. Manasria and I. Mudawar, "Analysis of the accuracy of the SIMPLE, SIMPLER, and SIMPLEC algorithms for the calculation of natural and forced convection," *ASME J. Heat Transfer*, vol. 117, no. 1, pp. 157–163, 1995.

[4] C. A. Acharya, "SIMPLER: A Simple Pressure Reformulation for the Calculation of Incompressible Flows," *ASME J. Fluids Eng.*, vol. 113, pp. 292–296, 1991.

[5] D. L. Sun, Z. G. Qu, Y. L. He, and W. Q. Tao, "An Efficient Segregated Algorithm for Incompressible Fluid Flow and Heat Transfer Problems—IDEAL (Inner Doubly Iterative Efficient Algorithm for Linked Equations) Part I: Mathematical Formulation and Solution Procedure," *Numer. Heat Transfer, Part B: Fundam.*, vol. 53, no. 1, pp. 1–17, 2007. doi: 10.1080/10407790701632543.

[6] J. Li, Q. Zhang, and Z.-Q. Zhai, "An efficient SIMPLER-revised algorithm for incompressible flow with unstructured grids," *Numer. Heat Transfer, Part B: Fundam.*, vol. 71, no. 5, pp. 425–442, 2017. doi: 10.1080/10407790.2017.1293965.

[7] D. S. Jang, R. Jetli, and S. Acharya, "Comparison of the PISO, SIMPLER, and SIMPLEC algorithms for the treatment of the pressure–velocity coupling in steady flow problems," *Numer. Heat Transfer, Part B: Fundam.*, vol. 10, no. 3, pp. 209–228, 1986. doi: 10.1080/10407798608552506.